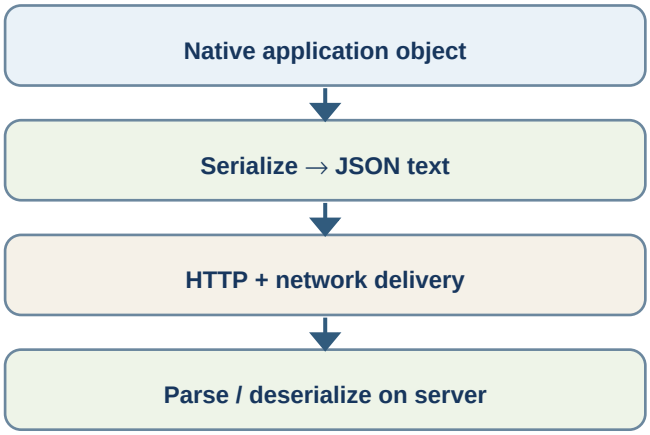
7. SERIALIZATION AND DESERIALIZATION

For Backend Engineers · Detailed Study Notes



Core idea: Serialization converts an application's native data into a standard format for transmission or storage. Deserialization parses that format back into a usable data structure at the receiving end.

Core idea: Serialization converts an application's native data into a standard format for transmission or storage. Deserialization parses that format back into a usable data structure at the receiving end.

1. The Core Problem: Language Barriers in Networking

A typical web application contains different systems:

- A frontend/client may be written in JavaScript.
- A backend/server may be written in Rust, Python, Java, Go or another language.
- The two sides may use different internal data structures, type systems and runtime representations.

Why is this a problem?

A JavaScript object is an in-memory structure understood by the JavaScript runtime. A Rust server cannot directly receive that JavaScript object as an in-memory object over the network. The two programs run in separate environments, often on different machines.

They therefore need an agreed format that both sides know how to produce and interpret.

Definition — Data interchange format: A documented representation of data that different systems can use to exchange information. Examples include JSON, XML, Protocol Buffers and Avro.

The shared format creates a boundary between the internal implementation of each program and the data exchanged between them.

2. What are Serialization and Deserialization?

Serialization

Serialization is the process of converting an application's in-memory data structure (such as an object, struct, list or class instance) into a defined format that can be transmitted or stored.

Example:

JavaScript object:

```
const user = {
  id: 101,
  name: "Asha",
  active: true
};
```

Serialized JSON text:

```
{
  "id": 101,
  "name": "Asha",
  "active": true
}
```

Deserialization

Deserialization is the reverse process: reading a serialized representation and converting it into a data structure that the receiving application can use.

For example, a backend may parse the JSON text and construct a Rust struct, Python dictionary or Java object.

Serialization vs Deserialization

Serialization	Deserialization
Converts native data into an interchange format	Converts the interchange format into native data
Usually happens before sending or storing data	Usually happens after receiving or reading data
Object / struct → JSON text	JSON text → object / struct
Also called encoding in some contexts	Also called decoding or parsing in some contexts

Important: Serialization is not the same as encryption or compression. Serialization defines how data is represented; encryption protects confidentiality/integrity, while compression reduces size.

3. Why Do We Need a Common Format?

A shared serialization format provides interoperability: systems can exchange data without needing to share the same programming language or internal object representation.

Benefits:

1. **Language independence:** A JavaScript client can communicate with a Rust, Python or Java server using the same documented format.
2. **Interoperability:** Different services can exchange structured data.
3. **Persistence:** Data can be serialized and saved to files, databases, caches or message queues.
4. **Predictable contracts:** Teams can agree on field names, types and structure.
5. **Easier debugging:** Text formats such as JSON can be inspected by humans.

A format does not automatically make every system compatible. Both sides still need to agree on the schema, field meanings, data types, versioning and validation rules.

4. Types of Serialization Formats

Serialization formats are commonly grouped into text-based and binary formats.

A. Text-Based Formats

Text-based formats represent data using readable characters.

Examples:

- **JSON (JavaScript Object Notation):** Widely used in REST APIs and web applications.
- **XML (Extensible Markup Language):** Tag-based format used in many integrations and document formats.
- **YAML (YAML Ain't Markup Language):** Human-friendly format commonly used for configuration.

Advantages:

- Easy for humans to read and inspect.
- Convenient for debugging.
- Often supported by many programming languages and tools.

Trade-offs:

- Can be larger than compact binary encodings.
- Parsing text can require more processing than decoding a compact binary representation, depending on the format and implementation.

B. Binary Formats

Binary formats encode structured data into compact binary representations rather than ordinary readable text.

Examples:

- **Protocol Buffers (Protobuf):** A schema-based binary serialization format developed by Google.
- **Apache Avro:** A schema-based data serialization system often used in data pipelines and distributed systems.

Advantages:

- Often compact, reducing network and storage size.
- Can be efficient to encode and decode.
- Schemas can make data contracts explicit.

Trade-offs:

- Not directly human-readable.
- Usually require schema definitions or generated code/tooling.
- Debugging may need specialized tools.

Text vs Binary Formats

Text-based	Binary
Human-readable representation	Encoded binary representation
Convenient to inspect manually	Usually needs tooling to inspect
Examples: JSON, XML, YAML	Examples: Protobuf, Avro
Often simpler for public web APIs	Often chosen for compactness or performance needs

No format is universally best. Choose based on interoperability, readability, performance, schema requirements and ecosystem support.

5. Deep Dive into JSON

JSON (JavaScript Object Notation) is a lightweight, text-based data-interchange format. Despite the word "JavaScript" in its name, JSON is language-independent and supported by many programming languages.

Common uses:

- Request and response bodies in HTTP REST APIs.
- Application logs.
- Configuration files.
- Data exchange between microservices and third-party systems.

JSON Syntax Rules

1. A JSON object begins with `{` and ends with `}`.
2. Object keys must be strings enclosed in double quotes.
3. A colon separates each key from its value.
4. Commas separate key-value pairs.
5. Strings must use double quotes.
6. Values can be strings, numbers, booleans, `null`, arrays or nested objects.
7. JSON does not support comments or trailing commas in standard JSON.

Example of a valid JSON object

```
{
  "id": 101,
  "name": "Asha",
  "age": 21,
  "isStudent": true,
  "skills": ["Python", "JavaScript"],
  "address": {
```

```
    "city": "Raipur",
    "country": "India"
  },
  "middleName": null
}
```

The six JSON value categories

JSON value	Example	Meaning
String	`"name": "Asha"`	Text enclosed in double quotes
Number	`"age": 21`	Integer or decimal number
Boolean	`"active": true`	`true` or `false`
Null	`"middleName": null`	Explicit absence of a value
Array	`"skills": ["Python", "Java"]`	Ordered list of values
Object	`"address": {"city": "Raipur"}`	Collection of key-value pairs

Common mistake: JSON is not identical to a JavaScript object literal. For example, JSON requires double-quoted keys and does not allow functions, `undefined`, comments or trailing commas.

6. The Backend Engineer's Mental Model: OSI Layers

When data travels over a network, it passes through multiple layers of networking protocols.

For application/backend development, it is useful to focus on the application-level representation and understand that lower layers handle delivery.

Simplified flow

```
Application data
      |
      v
JSON serialization
      |
      v
HTTP request body
      |
      v
Transport layer (e.g., TCP)
      |
      v
Network layer (IP packets)
      |
      v
Link / physical transmission
      |
      v
Network receives and reassembles data
      |
      v
HTTP body is made available to application
```

```
      |
      v
JSON deserialization
```

What happens underneath?

1. The application creates data and serializes it, for example into JSON.
2. HTTP carries the JSON representation in a request or response body.
3. Lower networking layers divide and encapsulate data into units such as TCP segments and IP packets (exact details depend on the protocol).
4. The data is transmitted over the network as signals representing bits.
5. At the receiving end, networking layers process and reassemble the data.
6. The HTTP implementation exposes the message body to the application.
7. The application parses/deserializes the body into usable data.

Backend engineer's abstraction: In ordinary application development, you generally work with the HTTP request/response body and the framework's parsing facilities. You do not manually convert JSON into electrical or optical signals.

The OSI model is a conceptual reference model; real Internet protocols are commonly described using the TCP/IP model. Do not assume every network uses TCP: HTTP/3, for example, runs over QUIC, which uses UDP.

7. Complete End-to-End Workflow

Consider a user submitting a registration form in a JavaScript frontend while the backend is written in Rust.

Step-by-step

1. **Client prepares data:** The frontend collects the user's name, email and other form values into a JavaScript object.
2. **Client serialization:** The frontend converts the object into JSON text.
3. **HTTP transmission:** The JSON text is placed in the HTTP request body, commonly with `Content-Type: application/json`.
4. **Network delivery:** Networking protocols transmit the request to the server.
5. **Server deserialization:** The backend parses the JSON and maps it into its own native data structure, such as a Rust struct.
6. **Validation and processing:** The backend validates the data, runs business logic and may save it to a database.
7. **Server serialization:** The backend converts the response data into JSON.

8. **HTTP response:** The server sends the JSON response to the client.
9. **Client deserialization:** The frontend parses the response JSON into a JavaScript object.
10. **UI update:** The frontend uses the resulting object to update the interface.

End-to-end diagram

```
FRONTEND (JavaScript)
Native object
     |
     | Serialize
     v
JSON request body
     |
     | HTTP + network transmission
     v
BACKEND (Rust / Python / Java)
JSON request body
     |
     | Deserialize / parse
     v
Native struct / object
     |
     | Validate + business logic
     v
Application result
     |
     | Serialize
     v
JSON response body
     |
     | HTTP + network transmission
     v
FRONTEND (JavaScript)
JSON response
     |
     | Deserialize / parse
     v
JavaScript object
     |
     v
Update UI
```

8. Practical Code Examples

JavaScript: Serialize and deserialize JSON

```
const user = {
  id: 101,
  name: "Asha",
  active: true
};

// Serialization: JavaScript object -> JSON string
const jsonText = JSON.stringify(user);

console.log(jsonText);
```

```
// Deserialization: JSON string -> JavaScript object
const parsedUser = JSON.parse(jsonText);

console.log(parsedUser.name);
```

- `JSON.stringify()` converts a JavaScript value into a JSON string.
- `JSON.parse()` parses valid JSON text into a JavaScript value.

Sending JSON using fetch

```
const user = {
  name: "Asha",
  email: "asha@example.com"
};

const response = await fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(user)
});

const result = await response.json();
console.log(result);
```

Here, `JSON.stringify()` serializes the request data. The browser sends it as the HTTP body. `response.json()` reads and parses the response body as JSON.

Python: Serialize and deserialize JSON

```
import json

user = {
    "id": 101,
    "name": "Asha",
    "active": True
}

# Serialization: Python dictionary -> JSON string
json_text = json.dumps(user)

# Deserialization: JSON string -> Python dictionary
parsed_user = json.loads(json_text)

print(parsed_user["name"])
```

- `json.dumps()` serializes a Python object into a JSON string.
- `json.loads()` parses a JSON string into a Python object.

9. Important Considerations and Common Pitfalls

- **Data validation:** Successfully parsing JSON does not guarantee that its values are valid for your application. Validate required fields, types, ranges and business rules.

- **Type differences:** JSON has a limited set of types. For example, dates are commonly represented as strings and need explicit parsing.
- **Unsupported values:** JavaScript values such as functions and `undefined` do not have direct JSON equivalents; serialization may omit or transform them depending on context.
- **Large payloads:** Very large JSON bodies can increase network transfer and parsing costs. Consider pagination, streaming or compact binary formats where appropriate.
- **Schema evolution:** If fields change over time, version or evolve the data contract carefully so old and new clients can interoperate.
- **Security:** Treat incoming data as untrusted. Validate it, enforce size limits and avoid unsafe deserialization mechanisms.
- **Serialization is not encryption:** JSON is readable text. Use HTTPS/TLS to protect data in transit.

10. Quick Revision Sheet

Term	Definition
Serialization	Converts native data into a standard format for storage or transmission.
Deserialization	Parses a standard format back into usable native data.
Data interchange format	Agreed representation used by different systems to exchange data.
JSON	Human-readable, text-based data-interchange format.
XML	Tag-based text markup format used to represent structured data.
YAML	Human-friendly text format commonly used for configuration.
Protobuf	Schema-based binary serialization format.
Avro	Schema-based data serialization system.
JSON.stringify()	Converts a JavaScript value into JSON text.
JSON.parse()	Parses JSON text into a JavaScript value.
json.dumps()	Converts a Python object into a JSON string.
json.loads()	Parses a JSON string into a Python object.
Schema	Definition of the expected structure, fields and types of data.

Most important takeaway

- **Serialization:** Native object → standard representation.
- **Deserialization:** Standard representation → native object.
- **JSON:** Common text-based format for HTTP APIs.
- **Binary formats:** Often chosen when compactness or performance is important.

- **Networking layers:** Deliver the bytes; application code works with the HTTP body and parses it.

11. Backend Interview Questions

1. **What is serialization?**

Converting an in-memory data structure into a standard representation for transmission or storage.

2. **What is deserialization?**

Parsing a serialized representation and converting it into a usable data structure.

3. **Why is serialization needed in backend systems?**

Different services and programming languages have different internal representations. A shared format allows them to exchange data.

4. **What is the difference between JSON and a JavaScript object?**

A JavaScript object is an in-memory value; JSON is a text format with strict syntax rules.

5. **What are examples of text-based serialization formats?**

JSON, XML and YAML.

6. **What are examples of binary serialization formats?**

Protocol Buffers and Avro.

7. **What does JSON.stringify() do?**

Converts a JavaScript value into a JSON string.

8. **What does JSON.parse() do?**

Parses JSON text into a JavaScript value.

9. **Does successful JSON parsing mean the input is valid for the application?**

No. The application must still validate required fields, types, ranges and business rules.

10. **Is serialization the same as encryption?**

No. Serialization defines data representation; encryption protects data against unauthorized reading or tampering.

11. **Why might a backend choose Protobuf over JSON?**

Protobuf can provide compact binary messages and schema-based contracts, which may be useful for performance-sensitive service communication.

12. **What is the role of the application layer in this workflow?**

It handles application protocols and data representations, such as HTTP messages containing JSON bodies.

Source video: 7. Serialization and Deserialization for Backend Engineers